MOHIT / NOVEMBER 2026

Your 30-minute DevOps roadmap

Small explanations. Real deployments. A clear way back after every break.

Goal: enter paid Linux, cloud operations, application support or DevOps trainee work, preferably onsite. Build towards AI infrastructure after acquiring operational experience.

Your starting point: strong interest in technology, Python confidence and Linux terminal familiarity; first practical Dockerfiles, cloud deployments, CI workflows and Terraform environments are still ahead.

What is inside	How to use it
24 study days / 144 technical blocks	Six 30-minute blocks per study day: 72 scheduled hours, including short breaks.
48 job-search blocks	Two blocks per study day: 24 scheduled hours. Apply while learning.
12 optional listening blocks	Use while cooking or as a passenger. Listening prepares or reinforces concepts; practical checks demonstrate skill.
Suggested main dates: 16 Nov - 12 Dec	Monday to Saturday. Sundays are optional catch-up, college or rest. Extend the plan when a prerequisite needs work.

Start with just the next block

Today is 4 November 2026. Your trip is 7-13 November; your first mid-sem exam is 15 November. Start with the short preparation plan on page 2. The main sprint starts after the exam.

A target, not a job guarantee

This is an entry-level practice syllabus, not complete DevOps mastery. Completion should leave you with a demonstrable deployment and troubleshooting evidence. Hiring time depends on demonstrated skills, eligibility, openings and interviews.

The project you will grow

Build a tiny Python API named **Pulse**: /health returns a status, /version returns a version, and configuration comes from environment variables. Add one small test and a tiny SQLite table only when the roadmap calls for them. Keep it local or use synthetic data.

Its path: Linux service, reverse proxy, container, cloud VM, tested build pipeline, controlled release, infrastructure code, monitoring, and a small Kubernetes lab. Each step adds one reason to learn the next.

Your calendar and ordinary day

Date	Required career work	College
4-6 Nov	Up to two blocks/day: create project/repo; inspect process and port; draft CV and 2-3 eligible applications.	Main study effort goes to exam preparation.
7-13 Nov	Travel/family week. Optional one listening/review block; no required new lab.	Keep exam preparation practical around travel.
14 Nov	Pause career work.	Revision.
15 Nov	Exam and recovery.	Exam.
16 Nov - 12 Dec	24 study days, six technical blocks and two job blocks per day. Sundays optional.	Reserve three hours initially; adjust to actual course load.

A movable daily template

Time	Activity
07:00-08:00	Wake, breakfast, basic chores
08:00-10:00	IITM mathematics / demanding coursework
10:15-11:15	Technical blocks B1 and B2
11:15-11:30	Longer break
11:30-12:30	Technical blocks B3 and B4
12:30-14:00	Cook, eat, rest. Optional audio for part of this time.
14:00-15:00	Technical blocks B5 and B6
15:15-16:15	Job blocks J1 and J2
16:15-17:15	IITM assignments / revision
Evening	Gym, travel, shower, cooking and dinner; shift other blocks around your gym time.
Later	Guitar / friends / quiet time, wind down; plan about eight hours for sleep.

The clock is flexible. Keep the order and daily outcomes. Six technical blocks = three hours; two job blocks = one hour. Do not count cooking audio as another hour of deliberate practice.

The rules that make blocks work

One block = 25 minutes of work + 5 minutes away

A useful default inside the 25 minutes: 3 minutes to identify the goal, 17 minutes to learn or act, and 5 minutes to verify. Pause safely at the boundary. If you are productively focused, join two blocks and take the break afterwards. Installation and downloads may need a separate setup block.

Type	Where	Completion test
L - learn	Desk or quiet reading	Explain one mechanism with an example.
P - practical	Laptop / lab	Show the command, configuration and observed result.
R - retrieve / review	Anywhere quiet	Explain yesterday's result before consulting notes.
A - audio	Cooking or passenger travel	Later, say one idea and one question. No terminal skill is claimed.
J - job search	Desk	An eligible application, follow-up, CV improvement or interview answer.

Restart protocol: preserve state, not constant stimulation

Before stopping, write three lines in NEXT.md: **Working now / Exact next action / Unanswered question.** Leave the next task small enough for a ten-minute start. At the next session, open this file before opening a feed.

A missed block is not a failed day. Continue at the next available block. After a missed day, do one 15-minute restart and resume the unfinished task. Never double tomorrow's workload to repay a missed day.

AI as a short tutor and lab reviewer

Ask for: one concept in under 150 words, one small example, one slightly harder task and one observable pass condition. Request hints first. Check configuration and commands against the linked official documentation. After help, change one detail and solve again yourself.

Predict the outcome before executing commands. Verify hypotheses with logs and tests. Use documentation freely; understanding and diagnosis matter more than memorizing every flag.

Keep interest without making tech a 24/7 obligation

Pick one question at the end of each session, such as: "Why does this container exit?" Save it for your next block. One optional audio block can keep you connected to the topic. Keep sleep, some conversation and enjoyable music; do not make deprivation or permanent urgency a requirement for progress.

Use the big monitor for the terminal/editor and the laptop for the current reference. Phone out of reach; one browser profile; disable autoplay. Put interesting detours in LATER.md. Measure useful work completed, not hours of background tech content.

Your resource shelf - pick one section

All links were checked or located on their publisher's site on 4 November 2026. Tutorials can change; follow current syntax. These are selected references, not a demand to finish every course. The roadmap exercises are original practice tasks.

R1 / Linux Upskill Challenge

Read only the named lesson: services (7), text/logs (8), networking (9), scheduling (10), users (13), permissions (14), archives (16), scripting (20). Use a local VM first.

R2 / Git: Recording Changes

Use the sections on status, staging, diff, commits and ignored files. The surrounding chapter covers remotes and undoing changes.

R3 / Python venv

Read the opening explanation and the commands for creating and activating an environment. Skip platform details that do not apply.

R4 / NGINX beginner guide

Use configuration structure, reload and simple proxy sections. Skip FastCGI for this project.

R5 / Docker: Run your first container

Follow one action at a time. For Dockerfile, storage, Compose and networking, use Get started linked from the Docker docs navigation.

R6 / Docker: Publishing ports

Read publishing versus exposing ports. Test one mapping locally.

R7 / AWS Educate

Choose Compute, Networking, Storage and Cloud Operations labs. These introductory labs do not require a credit card; they are not a permanent free server.

R8 / EC2 getting started

Follow launch, connect and cleanup. Read the current pricing/account conditions before creating paid resources.

R9 / GitHub Actions quickstart

Create the first workflow. Then inspect triggers, jobs, steps and logs.

R10 / GitHub Actions: Python

Adapt checkout, Python setup, dependencies and test execution to your tiny application.

R11 / Terraform: AWS tutorials

Use install/quick start, create, change, variables, outputs and destroy. Local Docker quick-start is an option before paid AWS work.

R12 / Kubernetes Basics

After core deployment works: deploy, explore, expose, scale and update. Use a local cluster; no paid managed cluster is needed.

Practice cards / 01-03

Each numbered item is one 30-minute block (25 work + 5 break). Use only the named resource section. A task may need a continuation block; preserve its ID and next action.

Day 01 / 16 Nov / Build your first runnable service

Reference: R2, R3

[ ] D01-B1 [P] Create a Git repo and NEXT.md; exclude .venv, .env and credentials. Result: clean git status after commit.

[ ] D01-B2 [L/P] Create and activate a Python virtual environment. Explain which Python executable is running.

[ ] D01-B3 [P] Use a tiny Flask or FastAPI app with /health. Install only its required packages; start locally.

[ ] D01-B4 [P] Request /health with curl. Record status, body, port and process ID.

[ ] D01-B5 [P] Stop the process, reproduce connection failure and restart. Explain why the error changed.

[ ] D01-B6 [R/P] Write a five-line start guide; recreate the environment in a fresh folder using that guide.

Day 02 / 17 Nov / Processes, users and a managed service

Reference: R1: lessons 7, 13, 14

[ ] D02-B1 [R] Explain yesterday's process, port and environment without notes; verify afterwards.

[ ] D02-B2 [P] Inspect the app with ps and ss. Identify its owner, listening address and port.

[ ] D02-B3 [P] Give a test file restrictive permissions; reproduce and fix a permission error without using chmod 777.

[ ] D02-B4 [P] Create a systemd unit for the app in a disposable Linux VM. Start and inspect its status.

[ ] D02-B5 [P] Set an incorrect executable path in the lab unit. Use status/journal logs to locate the failure.

[ ] D02-B6 [R/P] Correct the unit, reload it and restart. Save the unit and your diagnosis in the repo.

Day 03 / 18 Nov / Follow a request through the network

Reference: R1: lesson 9; R4

[ ] D03-B1 [R] Start your service and explain how its logs differ from terminal output.

[ ] D03-B2 [L/P] Compare localhost, a private IP and DNS. Resolve a hostname and inspect the answer.

[ ] D03-B3 [P] Use curl and ss to distinguish an HTTP error from a connection failure. Save both examples.

[ ] D03-B4 [P] Install Nginx in the lab and proxy one path to your app. Confirm the response.

[ ] D03-B5 [P] Point Nginx at a wrong upstream port. Diagnose using app and proxy logs; restore the port.

[ ] D03-B6 [R] Draw browser -> host/port -> proxy -> app. Explain where DNS and HTTP fit.

Practice cards / 04-06

Each numbered item is one 30-minute block (25 work + 5 break). Use only the named resource section. A task may need a continuation block; preserve its ID and next action.

Day 04 / 19 Nov / Logs, scripts and a small support drill

Reference: R1: lessons 8, 10, 20

[ ] D04-B1 [R] Without notes, list checks for an unreachable app; then compare with yesterday's diagram.

[ ] D04-B2 [P] Filter a lab log with grep, tail and a pipeline. Extract the lines around one failure.

[ ] D04-B3 [P] Write a Python health-check script with timeout, clear output and a nonzero failure exit code.

[ ] D04-B4 [P] Run it on a schedule in your VM. Verify the run time and captured result.

[ ] D04-B5 [P] Ask Ashwin to stop the lab service or change one setting. Diagnose without being told the cause.

[ ] D04-B6 [R/P] Write symptom, evidence, cause, fix and prevention in incident-01.md. Record a two-minute explanation.

Day 05 / 20 Nov / Containers and host ports

Reference: R5, R6

[ ] D05-B1 [R] Explain a Python process and VM; state what you think a container isolates, then check R5.

[ ] D05-B2 [L/P] Install/check Docker using its Linux instructions. Run an existing image and inspect its status.

[ ] D05-B3 [P] Publish a container port locally. Identify host port versus container port.

[ ] D05-B4 [P] Inspect logs, stop/start and remove only your named lab container. Explain its lifecycle.

[ ] D05-B5 [P] Use a wrong port mapping. Prove whether the app or mapping is the problem; fix it.

[ ] D05-B6 [R] Explain image versus container, and publishing versus EXPOSE. Save one working command.

Day 06 / 21 Nov / Your first Dockerfile

Reference: R5: Get started and image building

[ ] D06-B1 [R] Run yesterday's container and explain the port mapping from memory.

[ ] D06-B2 [L/P] Write a minimal Dockerfile for Pulse: base, work directory, dependency install, code and start command.

[ ] D06-B3 [P] Build a tagged image; run it; check /health from the host.

[ ] D06-B4 [P] Change /version to v2 and rebuild. Explain why changing a file outside a running container did not update it.

[ ] D06-B5 [P] Omit a required file or package in a lab build. Use build/runtime logs to diagnose the failure.

[ ] D06-B6 [R/P] Fix the image; add a .dockerignore. Explain each Dockerfile instruction without reading a tutorial.

Practice cards / 07-09

Each numbered item is one 30-minute block (25 work + 5 break). Use only the named resource section. A task may need a continuation block; preserve its ID and next action.

Day 07 / 23 Nov / Configuration and persistent data

Reference: R5: environment and storage guides

[ ] D07-B1 [R] Rebuild your app image from a clean checkout using your README.

[ ] D07-B2 [P] Read a harmless setting from an environment variable. Run the same image with two values.

[ ] D07-B3 [P] Write a test file inside a container; remove/recreate it and observe what survives.

[ ] D07-B4 [P] Repeat using a named volume. Verify persistence after recreating the container.

[ ] D07-B5 [P] Run with a missing required setting. Make the error explicit and correct the configuration.

[ ] D07-B6 [R] Explain image, container writable layer and volume. Keep secrets outside Git and images.

Day 08 / 24 Nov / Compose and a reproducible local stack

Reference: R5: Compose and networking guides

[ ] D08-B1 [R] Explain how the app's configuration and data survive different lifecycle events.

[ ] D08-B2 [P] Write a Compose file for Pulse. Start and stop the stack with one command.

[ ] D08-B3 [P] Add an Nginx service. Route requests using the app service name, not its changing IP.

[ ] D08-B4 [P] Add or inspect a health check. Observe the healthy/unhealthy state after stopping the app.

[ ] D08-B5 [P] Break a service name or environment variable. Read both services' logs and restore access.

[ ] D08-B6 [R/P] Have someone follow your README, or test from a clean directory. Record any missing instructions.

Day 09 / 25 Nov / Cloud compute and account boundaries

Reference: R7, R8

[ ] D09-B1 [R] Explain how your local stack would change on a remote Linux VM.

[ ] D09-B2 [L] In AWS Educate, identify compute, storage, region and network concepts. Choose the Compute lab.

[ ] D09-B3 [P] Launch/connect inside the lab. Identify private/public addresses and the allowed connection method.

[ ] D09-B4 [L/P] Identify identity permissions versus network rules. Explain why one cannot replace the other.

[ ] D09-B5 [P] Inspect SSH access and host identity in your lab; explain public versus private key usage.

[ ] D09-B6 [R] Write a resource list and cleanup plan. Choose Educate or a deliberately budgeted personal account for later work.

Practice cards / 10-12

Each numbered item is one 30-minute block (25 work + 5 break). Use only the named resource section. A task may need a continuation block; preserve its ID and next action.

Day 10 / 26 Nov / Your first cloud deployment

Reference: R8; R5

[ ] D10-B1 [R] Explain region, VM, security group and SSH key before opening the console.

[ ] D10-B2 [P] Provision a small eligible lab VM; allow only needed access. If lab access is limited, practise on a local VM.

[ ] D10-B3 [P] Install the runtime/container tooling and copy or clone only the demo code.

[ ] D10-B4 [P] Start Pulse and verify it on the VM itself before testing remote access.

[ ] D10-B5 [P] Troubleshoot remote access layer by layer: address, network rule, listening socket, proxy, application.

[ ] D10-B6 [R/P] Document setup and actual cloud evidence. Mark any local-only work honestly. Clean up unused resources.

Day 11 / 27 Nov / Access, storage and repeatable releases

Reference: R7: Storage/Networking; R8

[ ] D11-B1 [R] Redraw the actual request route to your VM; identify every access boundary.

[ ] D11-B2 [L/P] Inspect least-privilege access for your lab task. Keep private keys/tokens out of the repository.

[ ] D11-B3 [P] Use an S3 lab to upload/download a harmless file; explain object storage versus a VM disk.

[ ] D11-B4 [P] Release app version v2 with a recorded image/version reference. Confirm /version.

[ ] D11-B5 [P] Introduce a bad app configuration. Restore the previous known-good configuration and version.

[ ] D11-B6 [R/P] Write release and rollback steps. If using a domain, inspect TLS setup; otherwise record HTTPS as an explicit next task.

Day 12 / 28 Nov / Cloud support practice and cost cleanup

Reference: R7: Cloud Operations; R8: cleanup

[ ] D12-B1 [R] Explain your manual deploy and rollback process as an interview answer.

[ ] D12-B2 [P] Inspect CPU, memory and disk usage. Record normal measurements for your small app.

[ ] D12-B3 [P] Choose a harmless lab failure: stop the app or fill a small test file quota. Diagnose and recover.

[ ] D12-B4 [P] Review compute, disks, public IPs and storage you created. Remove unused billable resources.

[ ] D12-B5 [P] Reproduce a reported issue from a short symptom description, without knowing the cause.

[ ] D12-B6 [R/P] Publish incident-02.md and your cloud diagram. List verified skills and remaining gaps on your CV.

Practice cards / 13-15

Each numbered item is one 30-minute block (25 work + 5 break). Use only the named resource section. A task may need a continuation block; preserve its ID and next action.

Day 13 / 30 Nov / First CI workflow

Reference: R9, R10

[ ] D13-B1 [R] Explain the difference between testing a change and deploying it.

[ ] D13-B2 [L/P] Create a GitHub Actions workflow with a push/manual trigger and one simple command.

[ ] D13-B3 [P] Check out your repo, set up Python and install dependencies in the runner.

[ ] D13-B4 [P] Run a tiny test for /health. Deliberately fail it and inspect the failed step.

[ ] D13-B5 [P] Fix the test or app appropriately. Verify the workflow goes from red to green.

[ ] D13-B6 [R] Explain event, workflow, job, runner, step and exit code using your actual run.

Day 14 / 01 Dec / Build and publish an image

Reference: R9; Docker image publishing guidance

[ ] D14-B1 [R] Recreate the trigger and test-job outline from memory; consult docs for exact syntax.

[ ] D14-B2 [P] Add a Docker image build after tests. Verify a failing test prevents the build path as intended.

[ ] D14-B3 [P] Tag the image with a version or commit identifier; explain why a changing latest tag is ambiguous.

[ ] D14-B4 [P] Publish your demo image to a registry with minimally scoped workflow permissions.

[ ] D14-B5 [P] Investigate a deliberate lab permission or tag error using workflow logs; fix it without printing credentials.

[ ] D14-B6 [R/P] Pull the specific image in your lab and run /version. Save the build-to-runtime trace.

Day 15 / 02 Dec / Controlled deployment and rollback

Reference: R9; your manual deployment notes

[ ] D15-B1 [R] Explain image registry versus running server, and the existing manual rollback.

[ ] D15-B2 [P] Write a small deployment script that selects a version and checks /health.

[ ] D15-B3 [P] Add a manual deployment workflow for your lab. Use current secure authentication guidance; avoid unrestricted inbound SSH.

[ ] D15-B4 [P] For a local-only lab, manually run the deployment script with the CI-built image and label this limitation.

[ ] D15-B5 [P] Deploy a deliberately bad lab release; run the rollback and show restored /version and /health.

[ ] D15-B6 [R/P] Record how deployment is triggered, authorized, verified and reversed. Ask Ashwin to review the design.

Practice cards / 16-18

Each numbered item is one 30-minute block (25 work + 5 break). Use only the named resource section. A task may need a continuation block; preserve its ID and next action.

Day 16 / 03 Dec / Terraform locally first

Reference: R11: installation and local quick start

[ ] D16-B1 [R] Explain which infrastructure steps you currently do by hand.

[ ] D16-B2 [L/P] Install Terraform and use its local Docker quick-start. Identify provider and resource.

[ ] D16-B3 [P] Run init and plan. Predict every proposed change before applying.

[ ] D16-B4 [P] Apply, inspect state and compare the actual resource to configuration.

[ ] D16-B5 [P] Change one harmless attribute and review the next plan. Explain replacement versus in-place change if shown.

[ ] D16-B6 [R/P] Destroy only the tutorial resources. Exclude state, credentials and .terraform from Git; commit appropriate config/lock files.

Day 17 / 04 Dec / Terraform for a small cloud environment

Reference: R11: create, variables and outputs

[ ] D17-B1 [R] Explain configuration, provider, plan, state and real resources using yesterday's run.

[ ] D17-B2 [P] Define a small VM and narrow network access following the AWS tutorial; use a funded account only if costs are understood.

[ ] D17-B3 [P] Use variables for one changing input and outputs for a useful identifier.

[ ] D17-B4 [P] Review the plan, apply and verify the actual resource. If credentials are unavailable, continue the local provider lab.

[ ] D17-B5 [P] Change an input and predict the plan. Check the result before accepting it.

[ ] D17-B6 [R/P] Clean up the lab. Record which provider you actually used; no AWS claim for an unexecuted configuration.

Day 18 / 05 Dec / State, drift and rebuilding

Reference: R11: state, change, destroy

[ ] D18-B1 [R] Explain why deleting the Terraform state file is not equivalent to deleting cloud resources.

[ ] D18-B2 [L/P] Inspect state metadata without publishing sensitive values. Explain team storage/locking at a conceptual level.

[ ] D18-B3 [P] Make a small manual change to your disposable resource. Observe how the plan surfaces drift.

[ ] D18-B4 [P] Restore the declared configuration. Recreate the lab using documented inputs.

[ ] D18-B5 [P] Compare Terraform provisioning with your deployment script. Identify who owns each operation.

[ ] D18-B6 [R/P] Destroy disposable resources and verify cleanup. Add a reproducibility section to the README.

Practice cards / 19-21

Each numbered item is one 30-minute block (25 work + 5 break). Use only the named resource section. A task may need a continuation block; preserve its ID and next action.

Day 19 / 07 Dec / Monitoring and operational evidence

Reference: R1: logs/scripting; R7: Cloud Operations

[ ] D19-B1 [R] Choose three useful checks for your app: availability, latency and resource use.

[ ] D19-B2 [P] Extend the health-check script to record timestamp, latency, status and failure cause.

[ ] D19-B3 [P] Set a simple repeated-failure threshold that writes a visible alert in your lab.

[ ] D19-B4 [P] Stop the app; show failure detection, then restart and show recovery.

[ ] D19-B5 [P] Generate a small controlled burst of local requests. Observe response times and CPU; avoid testing other people's systems.

[ ] D19-B6 [R/P] Write what the monitoring catches, what it misses and one next improvement. Fancy dashboards are optional.

Day 20 / 08 Dec / Data restore and support fundamentals

Reference: R1: lesson 16; Python sqlite3 docs

[ ] D20-B1 [R] Explain why a healthy process does not prove its data is backed up.

[ ] D20-B2 [P] Create a tiny SQLite database with synthetic records; query and filter it using SELECT and WHERE.

[ ] D20-B3 [P] Create a consistent backup of this tiny lab database and restore it to a separate path.

[ ] D20-B4 [P] Verify restored row counts and sample data. Document a repeatable restore procedure.

[ ] D20-B5 [P] Investigate a test bad file path or permission issue and restore access.

[ ] D20-B6 [R/P] Record incident-03.md. Explain backup success versus verified restore success in two minutes.

Day 21 / 09 Dec / Kubernetes, only after the core works

Reference: R12

[ ] D21-B1 [R] Explain why Compose on one host differs from a cluster scheduler. Review the core readiness gate.

[ ] D21-B2 [P] Install a local kind or minikube cluster following its current prerequisites; reserve another setup block if needed.

[ ] D21-B3 [P] Deploy your image. Inspect deployment, pod, events and logs.

[ ] D21-B4 [P] Reach the app through a local service or port-forward; verify /health.

[ ] D21-B5 [P] Set an invalid image tag. Inspect events, correct it and observe recovery.

[ ] D21-B6 [R] Explain desired state, pod, deployment and service using only your lab.

Practice cards / 22-24

Each numbered item is one 30-minute block (25 work + 5 break). Use only the named resource section. A task may need a continuation block; preserve its ID and next action.

Day 22 / 10 Dec / Health, resources and safe updates

Reference: R12: scale/update; linked probe concepts

[ ] D22-B1 [R] Explain yesterday's image failure and the evidence that revealed it.

[ ] D22-B2 [P] Add a readiness check and observe the result when the app is unavailable.

[ ] D22-B3 [P] Set sensible small resource requests/limits for the lab; explain scheduling versus enforcement.

[ ] D22-B4 [P] Update the app image and inspect rollout status; practise undoing a lab update.

[ ] D22-B5 [P] Delete a lab pod and observe replacement. Explain why persistent data requires separate handling.

[ ] D22-B6 [R/P] Save manifests and a short demo. If the core deployment remains weak, use this day to fix it instead.

Day 23 / 11 Dec / Independent rebuild and interview proof

Reference: Your repository; R1-R12 as needed

[ ] D23-B1 [R] Describe your complete system in three minutes before opening notes.

[ ] D23-B2 [P] Use a fresh folder or VM to rebuild the local service/container from your README.

[ ] D23-B3 [P] Run the test/build workflow and select a known image version.

[ ] D23-B4 [P] Deploy, verify and reverse a version change in the lab. Identify any undocumented manual steps.

[ ] D23-B5 [P] Ask Ashwin or an AI tutor for an unseen lab failure. Diagnose, explain and verify the fix.

[ ] D23-B6 [R/P] Correct documentation; label local/cloud limits and actual measurements honestly. Record a three-minute demo.

Day 24 / 12 Dec / Hiring review and next experiments

Reference: Job descriptions; your evidence

[ ] D24-B1 [R] Answer Linux, networking, Docker and CI questions aloud; mark uncertain answers.

[ ] D24-B2 [P] Repeat your weakest practical task without a tutorial. Consult docs when necessary.

[ ] D24-B3 [J] Match your verified project evidence to three current trainee job descriptions.

[ ] D24-B4 [J] Update CV and pinned repo; remove unsupported claims. Include your ongoing IITM programme accurately.

[ ] D24-B5 [R] Do a mock interview and a troubleshooting exercise. Request specific feedback, not a general confidence rating.

[ ] D24-B6 [J] Review response rates and interview gaps. Plan the next seven days around evidence, with applications continuing.

Listening blocks that connect to the labs

Optional: one 25-minute segment plus a five-minute spoken recap. A longer episode can span several blocks. Listen during cooking or safe passenger travel when attention allows. Pause around knives, heat, traffic or anything requiring full attention. Do not use audio to fill every quiet moment.

For theory, use read-aloud on a named reference section or a short AI explanation checked against it. For code-heavy sections, use listening only to understand purpose; inspect syntax at the desk. No audio files are included in this roadmap.

A01 / before D02 / Process versus service - R1 lesson 7. What keeps an application running after you close a terminal?

A02 / before D03 / How a request reaches Python - R1 lesson 9 + R4. Say the route from hostname to proxy to application.

A03 / before D05 / Image versus running container - R5. Explain image, process and container in your own words.

A04 / before D07 / Configuration versus persistent data - R5 storage guidance. What disappears when a container is replaced?

A05 / before D09 / What a cloud VM provides - R7 Compute + R8 opening. What is AWS responsible for and what must you configure?

A06 / before D11 / Identity versus network access - R7 Networking. Why can an authorized user still fail to connect?

A07 / before D13 / CI versus deployment - R9 concepts. What does a green test run prove, and what does it not prove?

A08 / before D15 / Versions and rollback - Your release notes. Which exact artifact would you restore after a bad release?

A09 / before D16 / Declarative infrastructure and state - R11 introduction. Why does Terraform keep track of resource identity?

A10 / before D19 / Detection and recovery - Your incident notes. Which signal tells you users cannot reach the service?

A11 / before D21 / Desired state and scheduling - R12 introduction. What should happen when a pod disappears?

A12 / before D24 / Tell the story of one incident - incident-01/02/03.md. Explain symptom, evidence, cause, fix and verification.

Optional industry listening - one segment a week

Red Hat Compiler: select **Container Roundup** around Docker week. Scaling with Orchestrators fits Kubernetes week. Episodes may exceed one block; use a stopping point.

Kubernetes Podcast from Google: optional after the basic cluster lab. Choose one discussion matching a topic you already practised. Industry conversation builds context; it does not replace task practice.

Two job blocks per study day

J1 / 30 minutes: find 2-3 genuinely relevant openings and check ongoing-degree eligibility, location, pay information and actual duties. Apply to 1-2 strong matches with the most relevant project evidence. Some applications need longer; quality takes priority over a daily quota.

J2 / 30 minutes: rotate follow-ups, one personal referral request, CV improvements and a spoken mock interview. Aim for roughly 15-20 suitable applications per week when enough eligible openings exist. Keep a simple tracker: company, role, source, eligibility, date, contact, response, next action.

Stage	What you can show	Search terms
Days 1-4	Python, Linux lab, logs and a health-check script	Linux trainee; system administrator intern; application support trainee
Days 5-12	Containerized app, proxy and actual VM deployment	Cloud support trainee; CloudOps intern; DevOps intern
Days 13-24	Test/build pipeline, release/rollback and infrastructure lab	DevOps trainee; junior platform intern; cloud operations associate

Use Ashwin for specific feedback

Ask whether he can review twice a week for 20-30 minutes. Send the repo change first. You operate the terminal; he asks why, introduces one fault in your disposable lab, and reviews your explanation. Ask for introductions if he knows relevant hiring teams. Do not claim his projects as your experience.

Readiness gate before adding more tools

Can you start and inspect a Linux service, follow an HTTP request, diagnose one permission/network failure, build/run your own image, deploy a version, read CI logs and restore a working version? If not, use Kubernetes days as continuation days. Apply to roles matching what you can already demonstrate.

Choose a role you can live on and learn in

Prefer paid onsite roles with genuine technical duties, a named supervisor, manageable commute and exam-compatible arrangements. Ask about shifts and on-call. Compare take-home pay with actual local PG/shared-room quotes, food, transport, utilities, mobile and a modest buffer. Budget the separate one-time deposit and travel cost before accepting relocation. No city or workplace can be assumed safe from its name alone; verify accommodation, commute and the employer.

When the search needs adjustment

After around 20 well-matched applications, review eligibility and CV evidence if there are no responses. If interviews reveal specific gaps, practise those tasks. If the work offered is mostly generic call handling, ask how much access you will have to logs, Linux, scripts and engineering escalation before treating it as an infrastructure stepping stone.

A block card, a restart note and today

Copy this block template

ID / type / prerequisite:

One outcome:

One reference section:

Predict what will happen:

Do the task:

Evidence / verification:

One change or failure to try:

Status: guided / independent / can troubleshoot

Exact next action:

A short tutoring instruction to reuse

Teach me this one block: [ID and task]. I know Python and basic Linux but am new to deployment. Explain the mechanism in at most 150 words, connect it to what I know, then give one task with a visible pass condition. Let me attempt it. Give one hint at a time if I get stuck. Check commands against current official docs. Finish with one slightly harder variation and ask me to explain the result. Do not complete the entire lab for me.

Today: Wednesday, 4 November

Block S1: create the Pulse folder, Git repo and virtual environment. Write NEXT.md. Completion: Python runs inside the environment and your first commit contains no secrets or environment directory.

Block S2: start a tiny local Python /health service, request it with curl, then stop and restart it. Completion: you can explain which process listens on which port. If setup takes the whole block, record the next action and resume tomorrow.

Keep IITM preparation the main work before 15 November. These starter blocks can satisfy equivalent tasks on D01; use freed time for testing yourself, not extra scope.

Your learning log - use one line per block

ID	Evidence / result	Help used	Next action
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

The useful measure is: what can I now build, explain or fix? A half-finished block with a clear next action is recoverable progress. The plan is designed to resume after ordinary life interrupts it.